

Dependable Hospital System: Zero-Trust Medical Architecture and Lateral Movement Containment

Trevor Ritchie

CSCI 641 - Advanced Cybersecurity

2026-04-27

Abstract—This report presents an architectural design and practical implementation strategy for a resilient hospital network utilizing Zero-Trust principles and micro-segmentation. The primary objective is to isolate high-value medical assets, such as Electronic Health Record (EHR) databases and infusion pumps, preventing lateral movement from compromised public-facing portals. We first describe an idealized, comprehensive architecture designed for total visibility and automated response. We then detail a practical implementation of this design within a simulated network environment (GNS3), evaluating both the successes of network-layer containment and the limitations of application-layer identity spoofing. By implementing a "Default-Deny" posture and multi-vector monitoring, the design prioritizes patient wellness (availability) and HIPAA compliance (confidentiality).

Index Terms—Zero-Trust, Micro-segmentation, Hospital Security, Lateral Movement, pfSense, Suricata, GNS3, Open Policy Agent.

high-level architecture simulations [4]. This project builds upon these works by integrating identity-aware application proxies with network-layer firewalls to create an educational simulation.

I. INTRODUCTION

Modern healthcare environments face increasing threats from ransomware and sophisticated lateral movement attacks. Traditional perimeter-based defenses are insufficient because they assume trust once an entity is inside the network. This project applies concepts from Ross Anderson's *Security Engineering* [1] to create a "Default-Deny" architecture where trust is never assumed based on network location. The primary security goal is patient wellness, which requires maintaining clinical availability while ensuring HIPAA-mandated confidentiality. This report is structured in two parts: first, we present an idealized architectural design for a resilient hospital network; second, we detail a practical Minimum Viable Product (MVP) implementation in GNS3 and analyze its effectiveness.

II. BACKGROUND AND RELATED WORK

The transition to Zero-Trust Architecture (ZTA) in healthcare is driven by the severe consequences of clinical downtime and data breaches. The U.S. Food and Drug Administration (FDA) has increasingly emphasized explicitly defined security policies for medical device cybersecurity [2]. Previous studies have demonstrated the efficacy of using virtualized environments like GNS3 to simulate and study the security of critical infrastructure, such as power electronic devices [3], and the use of containerization for

III. PART I: IDEALIZED ARCHITECTURAL DESIGN

This section presents the comprehensive, idealized design of the Dependable Hospital System. This architecture represents the "gold standard" for resilience and containment, prioritizing visibility and automated response.

A. Logical Topology and Micro-segmentation

The topology utilizes VLAN-based micro-segmentation with a centralized pfSense cluster as the primary Policy Enforcement Point (PEP). This design ensures that all inter-zone communication is explicitly authorized and monitored.

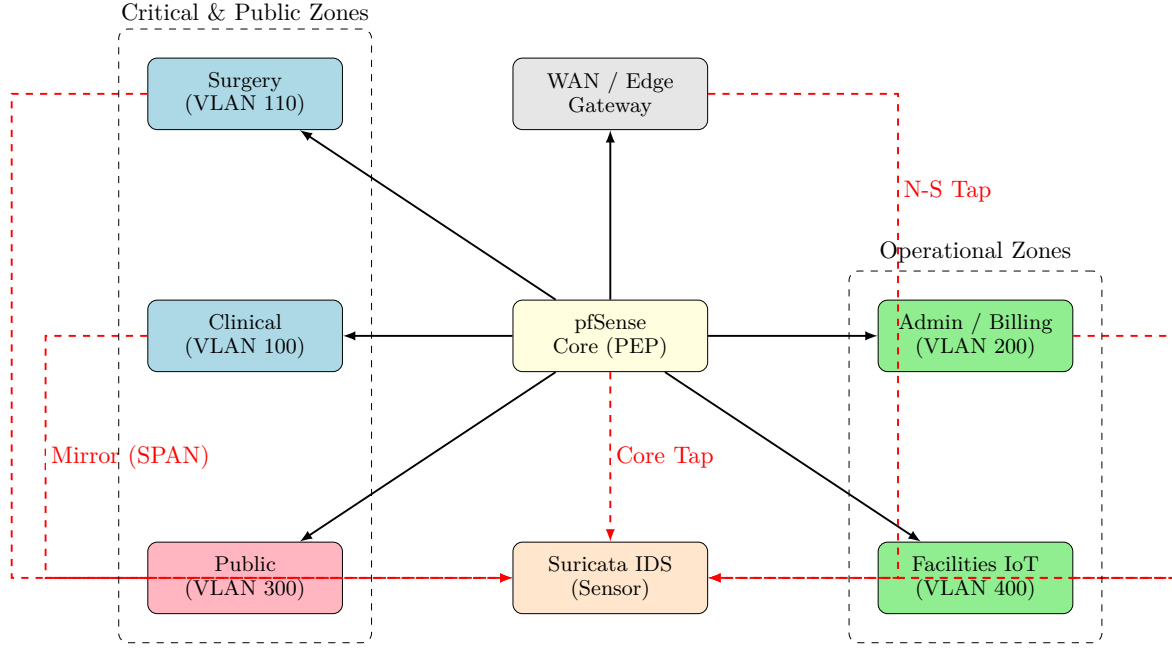


Fig. 1. Idealized Resilient Topology: Micro-segmentation and Multi-Vector Mirroring.

The architecture incorporates a Multi-Vector Mirroring strategy to ensure total visibility:

- **East-West SPAN (Clinical, Surgery, Public):** Captures traffic within the most sensitive zones to detect lateral movement attempts from Public Wi-Fi to clinical workstations.
- **Core Tap:** Monitored at the pfSense PEP, ensuring a central record of all cross-VLAN communication.
- **North-South Tap (Edge):** Monitors ingress exploits and egress exfiltration attempts (C2 traffic).

INSTALLED RULE SET MD5 SIGNATURES		
Rule Set Name/Publisher	MD5 Signature Hash	MD5 Signature Date
Emerging Threats Open Rules	9cb521bbe8e13e82d7f9937e3b7efed2	Thursday, 19-Mar-26 23:46:34 EDT
Snort Subscriber Rules	Not Enabled	Not Enabled
Snort GPLv2 Community Rules	8c630052516fe480fcf529e54d0fa414	Thursday, 19-Mar-26 23:46:34 EDT
Feodo Tracker Botnet C2 IP Rules	ce4793f2307aa4c0fdcf1ddfad224820	Thursday, 19-Mar-26 23:46:33 EDT
ABUSE.ch SSL Blacklist Rules	15d9e4a8fac051fecdc20d72dc88e9af	Thursday, 19-Mar-26 23:46:34 EDT
UPDATE YOUR RULE SET		

Fig. 2. IDS Configuration: Suricata configured on the pfSense Core with Emerging Threats (ET) open rulesets.

B. Zero-Trust Trust Boundary Model

Access is granted based on identity and device health rather than network location. Trust is never assumed, and every access request is explicitly evaluated by a centralized Policy Decision Point (PDP) before enforcement at a distributed Policy Enforcement Point (PEP).

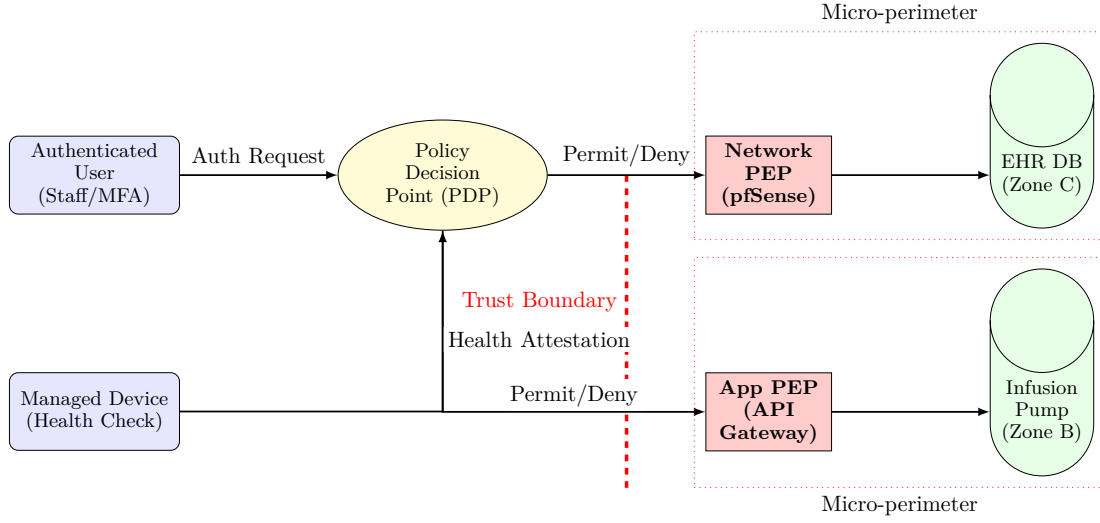


Fig. 3. Zero-Trust Trust Boundary Architecture: PDP/PEP Enforcement Model.

C. Incident Visibility and Response Sequence

The sequence (Fig. 4) illustrates the flow from initial detection to automated containment. An alert correlated by the SIEM triggers an automated quarantine command, isolating the actor via dynamic VLAN reassignment.

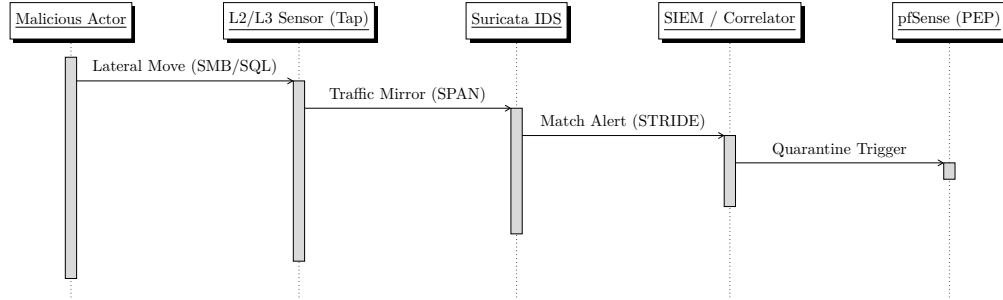


Fig. 4. Incident Attribution and Automated Response Sequence.

D. Policy Mapping and Advanced Threat Model (STRIDE)

The design is evaluated against the STRIDE threat modeling framework (Table I) to ensure architectural pillars mitigate common vectors in healthcare environments.

TABLE I
STRIDE THREAT ANALYSIS AND IDEALIZED MITIGATIONS

Threat	Description	Idealized Mitigation
Spoofing	Attacker impersonates a physician to access EHR data.	Identity-centric PDP with MFA and device health attestation.
Tampering	Modifies infusion pump settings.	Protocol-specific IDS and encrypted Modbus/TCP tunnels.
Repudiation	Denies lateral movement.	High-fidelity SPAN mirroring and SIEM attribution logs.
Info Disclosure	Ransomware exfiltrates HIPAA records.	Micro-segmentation and "Default-Deny" egress filtering.
DoS	Attack on Public Wi-Fi impacts Clinical VLANs.	Failure-containment zones and L2/L3 isolation.

E. Incident Containment and Response

Detection is distributed across the network (pfSense), protocol (Suricata DPI), and identity layers (SIEM). Upon detection, the system initiates:

- 1) **Dynamic Isolation:** Reassigning switch ports to a Quarantine VLAN.
- 2) **Session Termination:** Dropping all active connections associated with the compromised identity.
- 3) **Remediation:** Terminates and redeploys compromised containers from cryptographically signed images.

F. Clinical Continuity and Fail-Safe Logic

The architecture resolves the conflict between "Fail-Secure" and "Fail-Safe":

- **Data-Centric Assets (EHR):** Follow a *Fail-Secure* (Fail-Closed) model to prevent information disclosure.
- **Life-Critical Assets (Ventilators):** Follow a *Fail-Safe* (Fail-Open) model. The PEP defaults to a "Clinical Continuity" policy if the security infrastructure fails, ensuring patient treatment is never interrupted.

IV. PART II: PRACTICAL IMPLEMENTATION AND ANALYSIS

Time, resource, and skill-level constraints necessitated a scaled Minimum Viable Product (MVP) deployment within the GNS3 environment.

A. MVP Implementation Details

The practical implementation utilized a physical Spoke-and-Hub segmentation model:

- **Firewall Core (pfSense):** Central hub enforcing strict "Default-Deny" across physical interfaces.
- **Policy Decision Point (OPA):** Docker container running Open Policy Agent with Rego-based policies.
- **Policy Enforcement Point (Nginx):** Configured with `auth_request` to halt traffic and consult the OPA PDP.

B. Implementation Methodology: Troubleshooting-First

To ensure that security enforcement successes were not false positives caused by misconfiguration, the deployment followed a "Troubleshooting-First" methodology. Initial connectivity was established using temporary "Pass-All" rules across all physical interfaces. This allowed for the verification of basic routing, container availability, and ARP resolution. Once baseline functionality was confirmed via ICMP Echo Requests, the system was iteratively hardened into a "Strict Default-Deny" posture. This phase-based approach ensured that when a connection was later blocked (e.g., during the lateral movement demonstration), it was definitively due to an intentional security policy rather than a network-layer failure.

C. Architectural Planes: Security vs. Data

The MVP implementation makes a critical distinction between the user-facing *Data Plane* and the internal *Security Plane*.

- **Data Plane:** Traffic originating from the WAN (Attacker/User) targeting the Nginx Portal (Zone A). This plane is strictly restricted to HTTP (TCP/80).
- **Security Plane (Control Plane):** Encapsulated communication between the Nginx PEP (Zone A) and the OPA PDP (Zone C). A dedicated pfSense rule was implemented to allow only the Nginx node to reach the OPA sidecar on port 8081.

By isolating the Security Plane from the general user network, the architecture ensures that even if an attacker gains control of the public portal, they cannot interact with or influence the PDP's decision-making engine beyond the scope of a standard authentication sub-request.

D. Implementation Challenges and Technical Solutions

The implementation process revealed several critical technical hurdles that required iterative architectural adjustments.

1) *Virtualization Stack and Hypervisor Conflicts:* Initial attempts to utilize standard third-party hypervisors (VMware/VirtualBox) resulted in significant instability due to conflicts with Windows 11 Core Isolation (Memory Integrity) features. To resolve this while maintaining host security, the project pivoted to a native Linux KVM backend running within the Windows Subsystem for Linux 2 (WSL2). This allowed for hardware-accelerated nested virtualization for the pfSense and Docker nodes without compromising the host's primary security features.

2) *Transition to Physical Segmentation:* The MVP initially utilized VLAN-based logical segmentation. However, to achieve maximum hardware-level isolation consistent with high-assurance medical environments, the topology was re-architected to a "Spoke-and-Hub" model. The central switch was replaced with three dedicated physical zone switches, and pfSense was reconfigured with five independent physical adapters. This shift eliminated the risk of VLAN hopping and simplified the enforcement of "Default-Deny" policies at the physical boundary.

3) *Mitigating Identity Spoofing with JWTs:* As noted in the analysis, the initial reliance on plaintext HTTP headers for identity created a significant spoofing vulnerability. This was resolved by implementing a cryptographically secure identity system centered around an Identity Provider (IdP) model. In a production environment, an IdP (e.g., Keycloak or Okta) serves as a centralized authority that authenticates users and issues signed tokens. For this MVP, a mock Identity Provider was developed to simulate this issuance process.

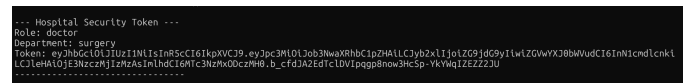


Fig. 5. Cryptographic Token Generation: Mock IdP issuing signed HS256 tokens.

The mock IdP allows for the manual generation of JSON Web Tokens (JWTs) signed with HS256, enabling the simulation of various identity contexts without the overhead of an enterprise authentication server. The OPA PDP was updated to utilize `io.jwt.decode_verify`, ensuring that access decisions are now bound to verified, tamper-proof claims rather than unauthenticated headers.

V. RESULTS AND ANALYSIS

A multi-stage attack was simulated against the GNS3 infrastructure.

A. Identity Enforcement: From Headers to JWTs

Initial tests demonstrated that a spoofed request using headers successfully retrieved protected data (Fig. 7), confirming a critical identity vulnerability.

```

root@webterm-2:~# curl -v http://10.0.10.10/ehr/
* Trying 10.0.10.10:80...
* Connected to 10.0.10.10 (10.0.10.10) port 80 (#0)
> GET /ehr/ HTTP/1.1
> Host: 10.0.10.10
> User-Agent: curl/7.88.1
> Accept: */*
>
< HTTP/1.1 403 Forbidden
< Server: nginx
< Date: Fri, 24 Apr 2026 22:16:04 GMT
< Content-Type: text/plain
< Content-Length: 47
< Connection: keep-alive
<
ZERO TRUST ERROR: Access Denied by PDP Policy.
* Connection #0 to host 10.0.10.10 left intact

```

Fig. 6. Anonymous request blocked with 403 Forbidden.

```

root@webterm-2:~# curl -v -H "X-User-Role: doctor" http://10.0.10.10/ehr/
* Trying 10.0.10.10:80...
* Connected to 10.0.10.10 (10.0.10.10) port 80 (#0)
> GET /ehr/ HTTP/1.1
> Host: 10.0.10.10
> User-Agent: curl/7.88.1
> Accept: */*
> X-User-Role: doctor
>
< HTTP/1.1 200 OK
< Server: nginx
< Date: Fri, 24 Apr 2026 22:16:08 GMT
< Content-Type: text/html
< Content-Length: 59
< Last-Modified: Fri, 24 Apr 2026 22:10:15 GMT
< Connection: keep-alive
< ETag: "69ebea47-3b"
< Accept-Ranges: bytes
<
ACCESS GRANTED: Patient EHR Records retrieved from Zone C.
* Connection #0 to host 10.0.10.10 left intact

```

Fig. 7. Spoofed header bypasses application-layer PEP.

However, the subsequent implementation of cryptographically signed JWTs successfully mitigated this risk.

```

root@webterm-1:~# curl -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.e
yJpc3MiOiJob3NwaXRhbC1pZHAiLCJyb2x1IjoIZG9jdG9yIiwiaWVudCI6InN1cmdlcnkiLCJleHAi
OjE3NzczMjIzMTc3MzZMx00czMhO.b_cfdJA2EdTcLdVIpqgp8now3HcSp-YkYwqIzE2Z23U" ht
tp://192.168.122.88/ehr/index.html
ACCESS GRANTED: Patient EHR Records retrieved from Zone C.
root@webterm-1:~#

```

Fig. 8. Verified Identity Access: Valid JWT granting 200 OK response from PEP.

Following the upgrade, the PEP (Nginx) and PDP (OPA) correctly rejected all unauthenticated and spoofed-header requests, only granting access when provided with a valid, signed token.

```

root@webterm-1:~# curl -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.e
yJpc3MiOiJob3NwaXRhbC1pZHAiLCJyb2x1IjoIZG9jdG9yIiwiaWVudCI6InN1cmdlcnkiLCJleHAi
OjE3NzczMjIzMTc3MzZMx00czMhO.b_cfdJA2EdTcLdVIpqgp8now3HcSp-YkYwqIzE2Z23U" ht
tp://192.168.122.88/ehr/index.html
<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx</center>
</body>
</html>
root@webterm-1:~#

```

Fig. 9. Identity Integrity: Invalid or tampered tokens result in immediate 403 Forbidden.

This verified the successful transition from a network-location-trust model to a robust, identity-centric verification model.

B. Containment of Lateral Movement

To evaluate the effectiveness of the physical micro-segmentation, a lateral movement attack was simulated originating from the compromised Nginx portal in Zone A. The adversary utilized **nmap** to scan for the Microsoft-DS (SMB) port (TCP/445) on a medical device within the Clinical Zone (Zone B). SMB scans are seen in Ransomware such as wannacry.

As shown in Fig. 11, the scan resulted in a **filtered** state. This confirms that the pfSense PEP intercepted and silently dropped the probe packets at the Zone A physical interface, successfully containing the breach and preventing the adversary from identifying or exploiting clinical assets.

×	2026-04-27 18:34:18.381086-04:00	ZONE_A_PUBLIC	Default deny rule IPv4 (1000000103)	10.0.10.10:46933	10.0.20.10:445
×	2026-04-27 18:34:17.381169-04:00	ZONE_A_PUBLIC	Default deny rule IPv4 (1000000103)	10.0.10.10:46931	10.0.20.10:445

Fig. 10. Defender Visibility: pfSense firewall logs showing the explicit drop of TCP/445 traffic from Zone A to Zone B.

```

Nmap done: 1 IP address (1 host up) scanned in 9.78 seconds
~ # nmap -Pn -p 445 10.0.20.10
Starting Nmap 7.97 ( https://nmap.org ) at 2026-04-27 22:33 +0000
Nmap scan report for 10.0.20.10
Host is up.

PORT      STATE      SERVICE
445/tcp    filtered  microsoft-ds

```

Fig. 11. Nmap Evidence: Lateral movement attempt blocked by pfSense PEP

C. Future Work: Advanced Context-Aware Authorization

While this project successfully implemented cryptographic identity integrity, a production-grade Zero-Trust environment would move beyond static roles to dynamic, context-aware authorization. In such a system, the Policy Decision Point (PDP) acts as a continuous risk engine, and the Policy Enforcement Point (PEP) is often distributed (e.g., using a Service Mesh like Istio or Envoy sidecars).

The issuance of a valid JWT would be contingent on a "Trust Score" derived from multiple real-time factors:

- **Device Health (Posture):** Verification that the accessing device is managed, fully patched, and has active endpoint protection.
- **Environmental Context:** Evaluation of the request's origin (e.g., internal clinical workstation vs. external VPN) and geolocation.
- **Temporal Constraints:** Aligning access rights with clinical shift schedules; a surgeon's access to EHR data might be restricted to their active duty hours.
- **Behavioral Analytics:** Identifying "impossible travel" or unusual data retrieval volumes that deviate from a user's historical baseline.

Transitioning to this "Signal-Based" trust model ensures that even a stolen cryptographic token is insufficient for access if the surrounding context indicates a high-risk scenario.

VI. CONCLUSION

This project demonstrates that Zero-Trust principles can effectively contain lateral movement in healthcare environments. By prioritizing patient wellness and implementing a context-aware failure model, the architecture provides a blueprint for modern, resilient hospital networks. The idealized design provides a robust framework for balancing clinical workflow needs with stringent network-layer isolation. The practical MVP implementation successfully verified that network-layer microsegmentation provides a critical safety net even when application-layer identity enforcement is compromised. Addressing the centralization of enforcement and moving towards cryptographic identity verification remain the primary paths for future architectural hardening.

VII. RESILIENCE AND FAILURE MODE ANALYSIS

A fundamental principle of dependable systems is predictable behavior during component failure. The architecture distinguishes between two primary failure modes based on the criticality of the protected asset.

A. Fail-Closed: Prioritizing Confidentiality (Zone C)

The administrative backend, housing the EHR database, operates on a *Fail-Closed* (Fail-Secure) model. If the Policy Decision Point (OPA) or the sidecar adapter becomes unavailable, the Nginx PEP is configured to intercept the resulting 500-series error and return a 403 **Forbidden**. While this impacts availability, it ensures that a failure in the security plane does not inadvertently create an unauthenticated bypass, thereby maintaining HIPAA-mandated confidentiality.

B. Fail-Safe: Prioritizing Patient Wellness (Zone B)

In contrast, life-critical clinical devices such as the infusion pump simulator in Zone B utilize a *Fail-Safe* (Fail-Open) network-layer model. While access to the device

is restricted by pfSense, the device does not sit behind an application-layer PEP that requires a real-time JWT verification for local clinical operation. This ensures that a latency spike or outage in the centralized identity plane cannot interfere with the immediate delivery of patient care, resolving the inherent conflict between security and clinical availability.

VIII. COMPLIANCE AND AUDITABILITY

The transition to a Zero-Trust architecture significantly enhances the organization's ability to meet regulatory requirements, specifically the HIPAA Security Rule's standards for access control and audit logs.

A. High-Fidelity Audit Trails

Traditional network-layer firewalls provide limited auditing capabilities, often restricted to source IP and destination port. By implementing an identity-aware PDP, the Dependable Hospital System generates high-fidelity JSON logs for every access attempt. These logs contain:

- The cryptographically verified **Identity** (Subject).
- The specific **API Resource** requested (Object).
- The precise **Timestamp** and **Decision** (Allow/-Deny).

This granular accounting allows for automated compliance reporting and proactive detection of insider threats or credential misuse, which are often invisible to perimeter-based defenses.

IX. TECHNICAL ARTIFACTS AND CONFIGURATION

The identity-aware security plane is implemented through a distributed chain of components, spanning the identity, application, and network layers.

A. Mock Identity Provider (Python)

The `mock_idp.py` script simulates an enterprise Identity Provider (Listing 1). It utilizes the `PyJWT` library to generate cryptographically signed tokens. The script includes functionality to simulate various security contexts, including valid role-based access, expired tokens (for time-based verification), and tampered signatures (to verify PDP integrity).

Listing 1. Hospital Mock Identity Provider (JWT Generator)

```
import jwt
import datetime

SHARED_SECRET = "hospital-zero-trust-secret-2026"
ISSUER = "hospital-idp"

def generate_token(role, department,
    expires_in_minutes=60, invalid_signature=
    False):
    payload = {
        "iss": ISSUER,
        "role": role,
        "department": department,
        "exp": datetime.datetime.utcnow() +
            datetime.timedelta(minutes=
                expires_in_minutes),
```

```

    "iat": datetime.datetime.utcnow()
}
# Simulation: Use a mismatched secret to
# produce an invalid signature
secret = SHARED_SECRET if not
invalid_signature else "wrong-secret"
return jwt.encode(payload, secret, algorithm=
"HS256")

```

B. Policy Decision Point (OPA Rego)

The OPA policy defines the "Source of Truth" for access control logic (Listing 2). Unlike traditional firewall rules that rely on IP addresses, this policy evaluates the role and path attributes extracted from the verified JWT.

Listing 2. OPA Rego Access Control Policy

```

package hospital.authz
default allow := false

# Access logic: Only allow if the role in the
# verified token is "doctor"
allow {
    [_, payload, _] := io.jwt.decode(input.token)
    payload.role == "doctor"
}

# Granular Rule: Allow patients to access only
# their specific record
allow {
    [_, payload, _] := io.jwt.decode(input.token)
    payload.role == "patient"
    input.path == ["ehr", "my-record"]
}

```

C. Policy Enforcement Point (Nginx)

The Nginx server acts as the primary PEP at the application layer (Listing 3). It utilizes the `auth_request` module to halt incoming traffic and initiate a sub-request to the security plane. A critical hardening measure is the use of `internal` locations, ensuring that the authorization endpoint is inaccessible to the public.

Listing 3. Nginx PEP Configuration (default.conf)

```

server {
    listen 80;

    location /ehr {
        auth_request /auth-check;
        proxy_pass http://ehr-database;
    }

    location = /auth-check {
        internal;
        proxy_pass http://10.0.30.20:5000/auth;
        proxy_pass_request_body off;
        proxy_set_header Content-Length "";
        # Forward original request details and
        # the JWT to the PDP
        proxy_set_header X-Original-URI
            $request_uri;
        proxy_set_header X-Original-Method
            $request_method;
        proxy_set_header Authorization
            $http_authorization;
    }
}

```

D. PDP Sidecar Adapter (Python/Flask)

The sidecar adapter (Listing 4) performs the necessary data transformation between the Nginx PEP and the OPA PDP. It extracts the Bearer token, constructs the JSON input expected by OPA, and translates the boolean decision into a standard HTTP status code (200 OK or 403 Forbidden).

Listing 4. PDP Sidecar Adapter

```

from flask import Flask, request, Response
import requests, json

app = Flask(__name__)
OPA_URL = "http://10.0.30.20:8181/v1/data/
hospital/authz/allow"

@app.route("/auth", methods=["GET"])
def auth():
    auth_header = request.headers.get("
Authorization", "")
    token = auth_header.split(" ")[1] if "Bearer
" in auth_header else ""

    input_data = {"input": {
        "token": token,
        "method": request.headers.get("X-Original-
Method", "GET"),
        "path": [p for p in request.headers.get("
X-Original-URI", "").split("/") if p]
    }}

    try:
        response = requests.post(OPA_URL, data=
json.dumps(input_data))
        if response.json().get("result") == True:
            return Response(status=200)
    except Exception:
        pass
    return Response(status=403)

```

E. Network-Layer Policy (pfSense)

To ensure the integrity of the security plane, pfSense enforces physical micro-segmentation. The network policy (Table II) follows a strict default-deny posture, with explicit rules only for the necessary security handshakes.

TABLE II
PFSense SECURITY PLANE RULE MAPPING

Interface	Source	Destination	Port	Purpose
WAN	Any	10.0.10.10	80	Public access to Nginx portal
Zone A	10.0.10.10	10.0.30.20	5000	PEP sub-request to PDP sidecar.
Zone C	10.0.30.20	10.0.30.20	8181	Local sidecar communication with OPA.
Any	Any	Any	Any	DEFAULT-DENY (Dropped and Logged).

X. AI DESIGN STATEMENT

I used Google Gemini via the Gemini CLI agent to orchestrate project tracks, convert my ideas into LaTeX, and keep track of progress in a process log where I would add screenshots. Most of all, I used it to provide step-by-step configuration tutorials for GNS3 and Alpine Linux nodes when I got stuck or things suddenly broke.

Example prompts include "How do I make the PDP/PEP more sophisticated than plaintext headers?" and many specific GNS3 troubleshooting prompts. The AI output was evaluated by empirically testing generated network configurations and Python scripts within the GNS3 simulation environment to verify behavioral correctness against the expected Zero-Trust security properties. My edits to the AI's output involved manually integrating, debugging, and refining generated code snippets for the OPA Sidecar and Nginx configurations to accommodate specific IP addressing and persistence constraints.

REFERENCES

- [1] R. J. Anderson, *Security Engineering: A Guide to Building Dependable Distributed Systems*, 3rd ed. Wiley, 2020.
- [2] FDA, *Cybersecurity in Medical Devices: Quality System Considerations and Content of Premarket Submissions*. U.S. Food and Drug Administration, 2025.
- [3] P. Nedyalkov, "Application of GNS3 to study the security of data exchange between power electronic devices and control center," *Computers*, vol. 12, no. 5, p. 101, 2023.
- [4] T. van den Berg et al., "Containerization of high level architecture-based simulations: A case study," in *Proc. 2017 ACM SIGSIM PADS*, 2017, pp. 71-82.